

Motion Planning and Experimental Validation for an Autonomous Bicycle

Alen Turnwald, Steven Liu

Abstract—Trajectory planning for an autonomous bicycle is investigated with regard to different vehicle models. To this end, a nonlinear optimization problem is set up using three models which differ with respect to the geometric details and, therefore, the included physical effects. First, using the method of *Direct Collocation* the optimization problem is transcribed with the vehicle models as equality constraints. The resulting trajectories are applied to the most detailed vehicle model using a 2-degree-of-freedom loop structure for validation and comparison. Further, a prototype autonomous two-wheeled vehicle is introduced which was developed for experimental verification of motion planning and control algorithms. Finally, experiments are presented and discussed that are run on the real vehicle for a particular maneuver. Thereby, the differences between the trajectories created by different vehicle models are emphasized.

I. INTRODUCTION

Autonomous riding of bicycles has been approached from different aspects. In this paper, we address the problem of optimal trajectory planning and experimental validation. The main objective is the investigation of trajectory planning methods using different vehicle models. Among others, a recently introduced model in [1] is considered and compared to a simpler model from [2] that neglects the geometric entities leading to the so-called *self-stability* of bicycles. The first mentioned model is suitable for controller design and includes the nonholonomic constraints. A nonlinear trajectory tracking controller using this model was presented in [3], the trajectory planning and experimental validations are the focus of this paper.

A. Methods of trajectory planning

We focus only on direct optimization-based trajectory planning in this paper that requires numeric solution of a nonlinear program (NLP). A survey on different approaches of numerical methods used for trajectory optimization can be found in [4]. [5] distinguishes between direct and indirect methods of solving transcribed nonlinear problems. Same categorization is found in [6] and other related literature. By transcription, an optimal control problem

$$\begin{aligned} \min_{\mathbf{u}(t), \mathbf{x}(t)} & \tilde{J}_b(\mathbf{u}_0, \mathbf{x}_0, \mathbf{u}_F, \mathbf{x}_F, t_0) + \int_{t_0}^{t_F} \tilde{J}_I(\mathbf{u}(t), \mathbf{x}(t), t) dt \\ \text{s. t. } & \dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t), \{\mathbf{x}(t), \mathbf{u}(t)\} \in \mathcal{C} \subseteq \mathcal{X} \times \mathcal{U} \\ & \mathbf{x}_0 = \mathbf{x}(t=0), \mathbf{x}_F = \mathbf{x}(t=t_F) \end{aligned} \quad (1)$$

A. Turnwald, and S. Liu are with the Department of Electrical Engineering, Institute for Control Systems, Technische Universität Kaiserslautern (TUK), Germany {turnwald, sliu}@eit.uni-kl.de

is converted into a nonlinear program (NLP) of the form

$$\min_{\mathbf{z} \in \mathbb{R}} J(\mathbf{z}) \quad (2)$$

$$\text{s. t. } \mathbf{b}_l \leq \begin{pmatrix} \mathbf{z} \\ \mathbf{C}(\mathbf{z}) \\ \mathbf{A}\mathbf{z} \end{pmatrix} \leq \mathbf{b}_u \quad (3)$$

that is, then solved numerically. J is the objective functional and (3) describes the generalized constraints. [5] distinguishes further between two transcription methods: *Shooting* and *Simultaneous*. First one uses an approximation of the input using predefined functions and numeric integration of the system dynamics to calculate the corresponding states. The latest, however, approximates both the input and the state of the systems and uses the equality constraints to satisfy the system dynamics at certain (collocation) points. The method used later in this paper, *Direct Collocation*, is an special case of the latest where the input is approximated as a piecewise linear function and the states as piecewise quadratic. Comparing to other description methods such as Direct Description for instance, Direct Collocation can reduce the number of decision variables by taking less collocation points. Also, the quadratic function for the states is in many cases, especially for mechanical systems, a better approximation of the nonlinear system equations than a linear function. More details on different transcription methods and their application in trajectory optimization can be found in [5]. Furthermore, one may distinguish between the methods for numerical integration of the system dynamics to include that in the optimization problem as a set of constraints. The trajectory planning discussed in this paper uses the *trapezoidal* integration that is defined by

$$\mathbf{x}_{t_N} = \int_{t_0}^{t_N} \mathbf{f} dt \Rightarrow \mathbf{x}_{t_N} \simeq \mathbf{x}_0 + \frac{1}{2} T_s (\mathbf{f}_{t_N} + \mathbf{f}_0) \quad (4)$$

The objective functional is also approximated using a sum

$$\min_{\mathbf{u}(t), \mathbf{x}(t)} \int_{t_0}^{t_N} \tilde{J} dt \simeq \min_{\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{x}_0, \mathbf{x}_1, \dots} \sum_{k=0}^{N-1} \frac{1}{2} T_s (\tilde{J}_k + \tilde{J}_{k+1}) \quad (5)$$

Trajectory planning for mobile vehicles has been a matter of research for a long time. Preliminaries on motion planning for mobile robots can be found e.g. in [7]. A good survey on the recent developments in motion planning for autonomous vehicles is given in [6]. [8] gives an overview on some key methods of model-based motion planning used for mobile robots. Research on trajectory planning for bicycles specifically has not been as extensive as those for mobile robots

or other vehicles, as also stated in [9]. Yet, many aspects are similar and can be reasonably transformed. [10] discusses general achievable trajectories for motorcycles and [11] uses a rigid body model for creating trajectories using dynamic inversions. [9] uses the nonlinear model from [2] to design optimal trajectories.

In this paper, we use a model derived based on the constrained Lagrangian, similar to [2] that is given by a set of Ordinary Differential Equations (ODE) and includes already the nonholonomic constraints. This allows to include the resulting ODE into the NLP directly without considering the nonholonomic constraints explicitly which simplifies the numeric solution of the NLP.

B. Review of the particular model and its use for planning

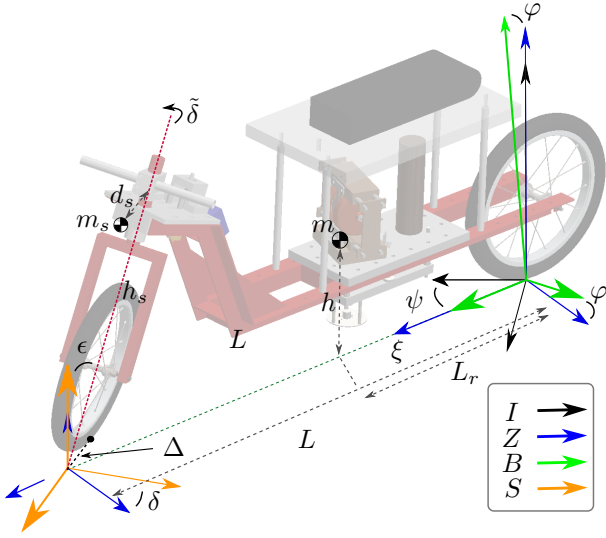


Fig. 1. Coordinate systems and parameters

The vehicle models used in the paper were described in [1]. We introduced a nonlinear model for the bicycle using the constrained Lagrangian L_r to include the non-holonomic constraints.

$$L_r(r^\alpha, s^a, \dot{r}^\alpha) := L(r^\alpha, s^a, \dot{r}^\alpha, \underbrace{-A_\alpha^a \dot{r}^\alpha}_{\dot{s}^a}). \quad (6)$$

$\dot{s}^a + A_\alpha^a \dot{r}^\alpha = 0$ describes the non-holonomic constraints with A being a *Ehresmann*-connection. r and s are the coordinates of the horizontal and vertical spaces correspondingly. Note that the generalized coordinates are given by $q^i = [r^\alpha, s^a]$. The variation of L_r yields the equations of motion the μ -th row of which is given as

$$\frac{d}{dt} \frac{\partial L_r}{\partial \dot{r}^\mu} - \frac{\partial L_r}{\partial r^\mu} + A_\mu^a \frac{\partial L_r}{\partial s^a} = - \frac{\partial L}{\partial \dot{s}^b} B_{\mu\alpha}^b \dot{r}^\alpha + \tau_\mu, \quad (7)$$

$$B_{\mu\alpha}^b := - \left(\frac{\partial A_\mu^b}{\partial s^a} A_\alpha^a - \frac{\partial A_\mu^b}{\partial r^\alpha} + \frac{\partial A_\alpha^b}{\partial r^\mu} - \frac{\partial A_\alpha^b}{\partial s^a} A_\mu^a \right). \quad (8)$$

τ_μ are the corresponding generalized forces. The states of the bicycle-system are

$$\mathbf{x} = [\mathbf{q}, \dot{\mathbf{r}}] = [\xi, \varphi, \delta, \psi, \eta, \quad \dot{\xi}, \dot{\varphi}, \dot{\delta}] \quad (9)$$

Fig. 1 illustrates the geometry, coordinate systems and system parameters of the bicycle.

The model considers detailed geometric features and physical effects such as the positive trail Δ that lead to the self-stability effect in two-wheeled vehicles. We call this model the *trailed* bicycle model. In Comparison, the so-called *naive* bicycle model, used e.g. in [2], ignores these effects leading to a less complex model. In [1], also a simplified model was introduced using approximations for containing tensors in the equations of motion. Later in this paper, we use these different vehicle models to investigate the trajectory planning methods. In particular, we use the differential equations derived by (7) in form of

$$M_{\mu\alpha}(r) \ddot{r}_\alpha = (\Gamma_{\mu\alpha\beta}(r) - \Upsilon_{\alpha\mu\beta}(r)) \dot{r}^\alpha \dot{r}^\beta + g G_\mu(r) + \tau_\mu. \quad (10)$$

where the tensors M, Γ, Υ and G contain information about the physics of the system. Thus, these are different for the naive and the trailed bicycle. The simplified model uses the Taylor-series to approximate those tensors, where for M and Υ the constant terms ($M(\mathbf{0})$ and $\Upsilon(\mathbf{0})$) and for Γ and G the first two terms or the linear approximation ($\tilde{\Gamma}(r)$ and $\tilde{G}(r)$) are used. The simplified model is then given by

$$M_{\mu\alpha}(\mathbf{0}) \ddot{r}_\alpha = (\tilde{\Gamma}_{\mu\alpha\beta}(r) - \Upsilon_{\alpha\mu\beta}(\mathbf{0})) \dot{r}^\alpha \dot{r}^\beta + g \tilde{G}_\mu(r) + \tau_\mu. \quad (11)$$

II. MOTION PLANNING BY DIRECT COLLOCATION

The considered scenario is a obstacle avoidance maneuver. Assume the vehicle moves on a pre-defined straight line and an obstacle is detected. The task of the motion planning is to adjust the path to navigate the vehicle safely around the obstacle and, then, end on the previous path.

A *path* $\mathbf{X}$ is a set of points on the x - y -plane. Since we use a collocation method and consider the ground track of the vehicle too. The decision variables are given by $\mathbf{z} = [\mathbf{x}, \mathbf{X}, \mathbf{u}]$ with $\mathbf{u} = [u_\xi, u_\varphi, u_\delta]$. u_φ , which is the torque directly applied to the leaning angle, is taken to be zero in the following. Note that the path variables are explicitly considered in the model and, therefore, in the optimization

$$\dot{x} = \dot{\xi} \cos \psi, \quad \dot{y} = \dot{\xi} \sin \psi. \quad (12)$$

As mentioned in the previous section, the direct collocation method uses linear approximation of the input and quadratic approximation of the states. This leads necessarily to a deviation from the physical behavior for complex systems. For a naturally unstable system such as the bicycle, this may lead to a large deviation of the states, once the resulting optimal input is applied in an open loop. For this reason, we use the structure illustrated in Fig. 2 for verification of the trajectories. We solve the optimization problem created using a vehicle model and, then, apply the input $\mathbf{u}^*$ as a feed-forward in addition to a linear feedback using the deviation from the optimal states $\mathbf{x} - \mathbf{x}^*$ to the trailed model. Note that this feedback is a simple linear controller and not designed to track the desired state, rather just to prevent the bicycle from falling over during the maneuver. We consider three different

vehicle models: The trailed model is the most detailed and, thus, the most complex model. The naive model neglects some physical effect and is simpler. We also use a simplified model that is created by simplification of the naive model as in (11) ([1]).

As illustrated in Fig. 3, a predefined nominal path $\mathbf{X}_{Nom}$ is assumed to be given as

$$\mathbf{X}_{Nom} = \{x \in [0, 100], y = 2\} \quad (13)$$

that is riding on a straight line. Suppose that an obstacle is detected and the path is modified to $\mathbf{X}_0$, called *Given path* in Fig. 3 as

$$\mathbf{X}_0 = \{x, y \in \mathbf{X}_{Nom} | y = 5 \forall x \in [33, 40]\} \quad (14)$$

The number of collocation points are taken to be $N = 25$. The initial states are

$$\dot{\mathbf{x}}(0) = 3.7 \text{ m/s}, \mathbf{q}(n=1) = \mathbf{0}, \mathbf{X}(0) = \mathbf{X}_0 \quad (15)$$

and the terminal values

$$\dot{\mathbf{x}}_F = 4 \text{ m/s}, \psi_F = 0 \quad (16)$$

The transcribed system dynamics from (7) by trapezoidal method is taken as the nonlinear constraint where for each model, the corresponding tensors from [1] are used. The objective functional is defined by

$$\begin{aligned} J_P &= K_p \sum_{n=1}^{N-1} (\mathbf{X}(n) - \mathbf{X}_0(n))^2 + \\ &K_{p,F} (\mathbf{X}(N) - \mathbf{X}_0(N))^2 + K_{\psi,F} (\psi(N) - \psi_F)^2 \\ J_u &= K_u \sum_{n=1}^N \mathbf{u}(n)^2 \\ J_c &= K_{c,\varphi} \sum_{n=1}^N \varphi(n)^2 + K_{c,\delta} \sum_{n=1}^N \delta(n)^2 \\ J_d &= K_{d,\dot{\varphi}} \sum_{n=1}^N \dot{\varphi}(n)^2 + K_{d,\dot{\delta}} \sum_{n=1}^N \dot{\delta}(n)^2 \\ J &= J_P + J_u + J_c + J_d \end{aligned} \quad (17)$$

The objective functional contains different terms. The first one J_P penalizes the distance to the given path weighted by $K_p > 0$. It also includes the path terminal costs weighted by $K_{p,F}$ as well as the terminal cost on the vehicle orientation weighted by $K_{\psi,F}$. The last term makes sure that the vehicle orientation is as desired at the end of the maneuver. This is even more important once trajectory planning is supposed to run in a cyclic manner due to the nonholonomic constraints on the vehicle kinematics. A wrong vehicle orientation at the beginning of the next cycle may lead to undesired trajectories or, in worst case, infeasibility of the follow-up problem. J_u penalizes the input minimizing the input effort. J_c penalizes the leaning angle and the steering angle to prevent the bicycle from falling over and J_d is a measure for riding comfort and prevents sudden change of the leaning or steering angles

providing smoother functions for those angles. The lower and upper bounds for the variables $\mathbf{z}_n$ are taken to be

$$\begin{aligned} \mathbf{z}_l &= [\mathbf{x}_l, \mathbf{X}_l, \mathbf{u}_l], \mathbf{z}_u = [\mathbf{x}_u, \mathbf{X}_u, \mathbf{u}_u] \\ \mathbf{x}_l &= \begin{bmatrix} 0 & -\frac{\pi}{3} & -\frac{\pi}{2.1} & -\pi & -10^{-16} & 3.7 & -20 & -20 \end{bmatrix} \\ \mathbf{x}_u &= \begin{bmatrix} \infty & \frac{\pi}{3} & \frac{\pi}{2.1} & \pi & 10^{-16} & 4.5 & 20 & 20 \end{bmatrix} \\ \mathbf{X}_l &= \begin{bmatrix} -5 & -1 \end{bmatrix}, \mathbf{X}_u = \begin{bmatrix} 105 & 6 \end{bmatrix} \\ \mathbf{u}_l &= \begin{bmatrix} -2 & 0 & -0.2 \end{bmatrix}, \mathbf{u}_u = \begin{bmatrix} 2 & 0 & 0.2 \end{bmatrix} \end{aligned} \quad (18)$$

Note that the constraints on η are taken to be *almost zero* (10^{-16}) to make sure that the initial guess strictly satisfies all constraints and the solver (in our case `fmincon`) does not throw warnings or errors. Also, the middle term in $\mathbf{u}_l$ and $\mathbf{u}_u$ that corresponds to u_φ is taken to be constantly zero. This is caused by the under-actuated nature of the bicycle. There are already approaches to add a direct torque to the leaning angle using different actuators such as CMGs (e.g. [12], [13], [14]), in this paper however, we suppose a conventional bicycle. We use the Matlab-toolbox introduced in [5] to solve the NLP using the following parameters

$$\begin{aligned} K_u &= 10, K_P = K_{p,F} = 8, K_P = K_{\psi,F} = \frac{8 \cdot 180}{\pi} \\ K_{d,\varphi} &= K_{d,\delta} = 20, K_{c,\varphi} = K_{c,\delta} = 1 \end{aligned} \quad (19)$$

The resulting trajectories are optimal and satisfy the requirements at the collocation points, yet not along the entire vehicle motion. The results are illustrated in Fig. 3. One can see that the optimization results are very similar for all three models (Opt by *). The optimization time is, however very different. The time required to solve the problem with the trailed model is much higher than the one with the naive and the shortest time was required for the problem with the simplified naive model. This comparison is legitimate since all problems were run under the same condition and on the same computer. Note that the simplified naive model contains tensors of at highest second order in terms of the states. Once the trajectories are applied to the realistic bicycle model for simulation, it becomes obvious that the drawback of the low optimization time is the error in the input and the states. As shown in Fig. 3, the trajectories created by the trailed model has the smallest deviation from the original path. Note that the difference between the naive model and the simplified naive model is very small. A similar pattern were found out by our experimental investigations described in the next section.

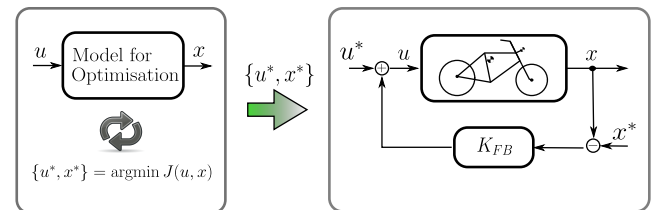


Fig. 2. Schematics of the trajectory optimisation for the bicycle

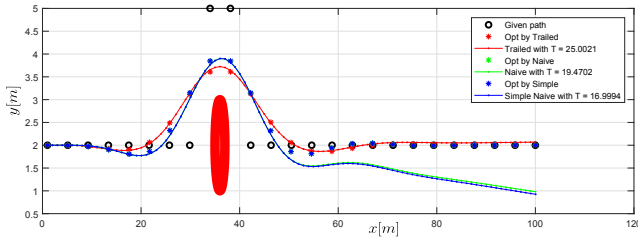


Fig. 3. Results of the trajectory optimization for the bicycle by direct collocation

III. PROTOTYPING AND IMPLEMENTATION

For experimental validation of our algorithms, we used a bicycle developed at the institute for control systems, TUK. The vehicle was designed and developed at the university as an experimental test bench for real-world investigations of motion control methods. A detailed description of the vehicle is out of scope of this paper, yet some key features are mentioned below. The main system components and the communication diagram is illustrated in Fig. 4.

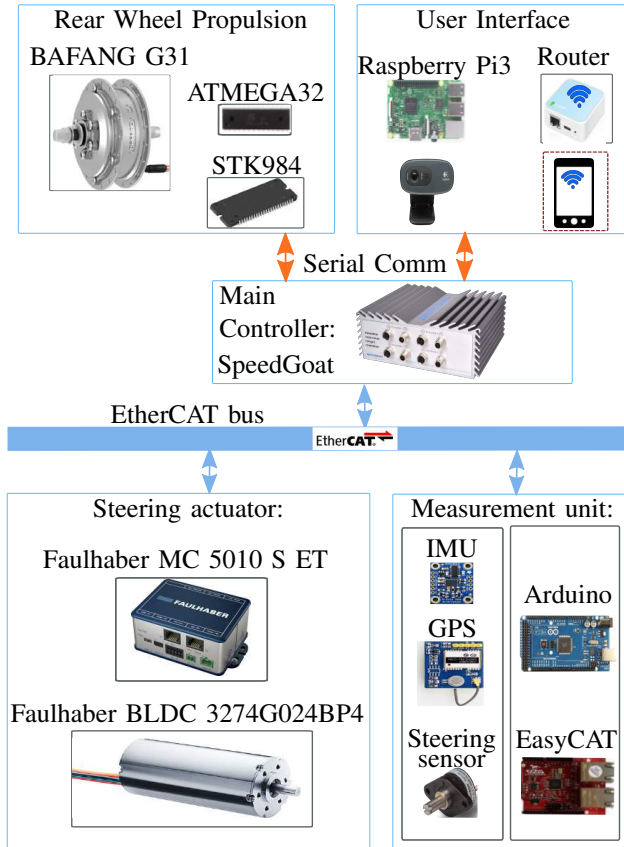


Fig. 4. System components and communication diagram

The main communication bus of is EtherCAT. A *Baseline-S* real-time target machine from *Speedgoat*[®] was chosen as the main processor with *Simulink Real-Time*[®] as operating system. The Speedgoat runs all the motion control algorithms of the bicycle. It was also used as the EtherCAT master, for communicating with other subsystems with EtherCAT slave

controller as well as data logger. Also, a state machine is implemented for initialization, safety check, sequential start-up/shutdown of subsystems and management of the data from and to the user interface.

To retain the self-stability of the bicycle which was among others matter of our research, a belt system was chosen to connect the steering motor directly (withough a gear box) to the steering axis. This helps the steering axis rotate almost freely when the motor is unpowered. Since for this purpose high torque picks are required, we used a *Faulhaber* BLDC 3274G024BP4 with 150W and a stall torque of 2697 mNm. As driver, Faulhaber *MC 5010 S ET* Motion Controller was used which has an EtherCAT interface. For the rear wheel propulsion a *BAFANG* FM G31 motor with a rated power of 250W and maximum torque of 30 Nm was chosen which is commonly used in many E-bikes. The driver was developed in our lab using an integrated power module *STK984-090A-E*. The control algorithms as well as a special serial communication protocol are hosted on an ATMEGA32.

The measurement unit contains different sensors such an IMU (BNO055) to measure the leaning and orientation angles, a GPS module (NEO 7M-C) and a hall sensor (ENS22F) for accurate measurement of the steering angle as well as its rate. All sensor data are collected on an *Arduino Uno*[®] and passed to the EtherCAT bus using an *EasyCAT*[®] shield that provides an EtherCAT slave controller communication to the Arduino via SPI interface. The User Control Unit (UCU) accepts inputs from the user and sends the gathered input data to the Speedgoat. Also, the UCU reads status information from the Speedgoat and updates it back to the user. For this purpose, a *Raspberry Pi3*[®] is used to run a Python script providing a web server on which the user can choose different scenarios and start/stop experiments via WiFi on a front-end device. A small router sets up a local WiFi network. Also, images from the camera installed in the bicycle are streamed to the user vie the same interface.

Up to 10 scenarios are saved as .dat-files on Speedgoat and can be run once the user chooses one. To run more scenarios, a host computer needs to be connected to Speedgoat (Ethernet) to push new files to its memory. A Scenario contains a desired state vector x^* , a desired input vector u^* as well as an *Enable* signal that is a boolean to mark the beginning and end of the scenario.

IV. CIRCULAR RIDES AND EXPERIMENTAL SETUP

For the experimental investigation presented in this paper, we created trajectories to make the bicycle go in a circle after a short period of straight riding and, end up on a straight line. We used the Naive model and the Trailed model for a comparison. We run four different variations of the same experiment distinguished by

- 1) whether the desired inputs (u_δ^* , u_ξ^*) and states (x^*) were generated by the naive or the trailed model, and
- 2) whether the system inputs are calculated only by a feed-back of the desired states or desires inputs were added as a feed-forward.

TABLE I
PARAMETER USED FOR TRAJECTORY GENERATION (ALL SI UNITS)

Parameter	Value	Short description
$(J_x, *, J)$	$(7.16, *, 18.04)$	Inertia of rear frame
$(J_{sx}, *, J_s)$	$(0.1071, *, 0.0852)$	Inertia of steering
$(h, 0, L_r)$	$(0.45, 0, 0.5)$	CoM of rear frame
$(h_s, 0, d_s)$	$(0.6, 0, 0.015)$	CoM of steering
m	50	Mass of the rear frame
m_s	9	Mass of the steering
J_w	0.163	Inertia of front wheel
Δ	0.09	Trail
R	0.388	Radius of front wheel
g	9.81	Gravity constant
L	1.25	Wheelbase
ϵ	19°	Head Angle

For creating the trajectories, we use the system parameter from the Table I in our model. Note that these values are partly measured and partly estimated using simplified geometric relations. The structure of the control loop is the same as given in Fig. 2. However, in case of feed-back only, we cut off the connection from u^* . The system states are mainly read directly out of the sensing unit. For the leaning angel φ we use the roll-component of Euler-angle provided by the IMU. The corresponding rate $\dot{\varphi}$ is directly read from the gyroscope. The steering angel δ is measured by a hall-effect sensor and the rate $\dot{\delta}$ is read out of the Faulhaber-driver. The forward velocity v is calculated within the driver and sent over the serial communication. One can see, however, that there are several "jumps" in the measurement that is due to the way of calculation using the time passed between two hall sensor signals of the BLDC.

The feedback controller was originally planned to be the one introduced in [3] (Eq.22) that is a passivity based tracking controller shaping the energy of the closed loop. Since our measured generalized velocities were not accurate enough, we decided to ignore the shaping of the kinetic energy that uses the desired and actual rates ($\dot{r}^*$ and $\dot{r}$). Instead, we used a simpler feedback law. Yet for comparison to [3], our feedback includes the controller corresponding to the virtual potential β_c and the damping u_d that are

$$\beta_c = G^{-1} J_{12}^T \frac{\partial^T H_c(q, p)}{\partial q}, \quad J_{12} = \begin{pmatrix} I \\ -A \end{pmatrix}$$

$$H_c = \frac{1}{2} \bar{q}^T \begin{bmatrix} 100 & 0 & 0 & 0 & 0 \\ 0 & 21.8 & 0 & 0 & 0 \\ 0 & 0 & 4.7 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \bar{q} \quad (20)$$

$$u_d = - \begin{bmatrix} 350 & 0 & 0 \\ 0 & 2.6 & 0 \\ 0 & 0 & 0.26 \end{bmatrix} \cdot (\dot{r} - \dot{r}^*) \quad (21)$$

Note that the theoretical proof of the stability of the tracking is no more valid in the experimental results. We repeated every experiment at least three times to make sure that the recorded data and the resulting conclusions are consistent.

The circle ride experiments were run on the football court of the university. On one hand, the free space enabled us to run our experiments and, on the other hand, the grass ground caused large disturbances on the measurements of the sensors which we took as a challenge. As shown in Fig. 5, for consistency we started every experiment from the same point with the bicycle (as far as possible) in the same initial orientation. Fig. 5 also includes 4 shots of the various videos¹ we recored. We start every experiment by holding the bicycle in the upright position after calibrating the sensors and defining the offsets. The bicycle, then, starts the experiments by a command from a mobile phone and ends it either with another command or after the saved trajectory is played back to the end.

The resulting ground track can be seen in Fig. 6. The leaning angle φ , steering angle δ , forward velocity v as well as the demanded input torque on the steering angle u_δ are shown in Fig. 8 for the trajectories created by the naive model and in Fig. 7 for the trajectories created by the trailed model.

Note that in Fig. 6 we defined the south-east corner of the football court as the origin of our world coordinate systems. The starting point is marked as a red point. One can see that both ground tracks without feed-forward (Naive:fb and Trailed:fb) result in large circles which means that the desired leaning angle φ^* is not tracked well. Adding feed-forward improves the tracking of the leaning angle and therefore the circular ground track. In this case, the main difference between the trajectories created by the two models can be explained. As one can see in Fig. 7, the feed-forward steering torque (ff) is nonzero during the circular motion in contrast to that from Fig. 8 which is an intuitive proof of the fact that the trailed model creates more realistic trajectories. Besides, although the feed-back torque in addition to the feed-forward (fb i.a.t. ff) seems similar, the change of the leaning angel as well as its tracking is much more natural in case of the trajectories from the trailed model. This fact becomes more clear looking at a zoomed area around 10s where the leaning angle in Fig. 7 changes much smoother to the desired value that from Fig. 8. The same effect can be observed for the steering angle which has a smoother transition in Fig. 7.

V. CONCLUSION AND FUTURE WORK

The main focus of this paper is on the motion planning for an autonomous bicycle with respect to different vehicle models. First, a review of the methods used for trajectory optimization as well as vehicle modeling is given. Then, the method used in this paper, namely *Direct Collocation* is described for transcription and creating a nonlinear program. Three vehicle models are considered, each describing the bicycle in another depth of physical details. The trajectories created are applied to the most complex model for comparison. Also, experimental investigations are run to compare the trajectories created by different models. A prototype

¹Please watch our video under youtu.be/d92tF75iQmk. From the minute 2:13 on the experiments illustrated in Fig. 6 are played.

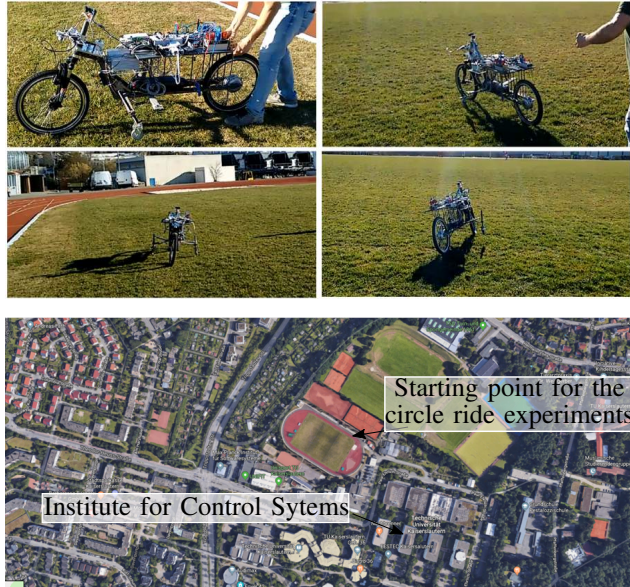


Fig. 5. Recordings of the circle ride. (GPS: 49.4269556°, 7.7529987°)

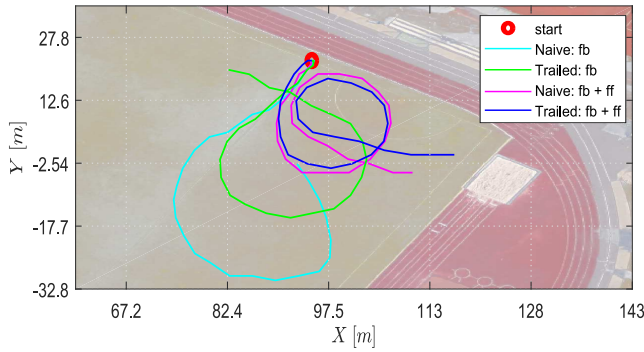


Fig. 6. Gourd paths on the football court

vehicle is introduced to which the trajectories are applied. It is concluded in both simulation and the real experiment that the trajectories created by the detailed vehicle model, the so-called trailed model, are more proper for motion control tasks creating a more natural system behavior. However due to complexity, the numerical solution of the optimization problem will take longer. As a future work, more complex trajectories for more real-life scenarios could be designed and compared on the experimental vehicle. Also, with regard to the accuracy and computation time on-line cyclic planing and control of the vehicle can be investigated.

REFERENCES

- [1] A. Turnwald and S. Liu, "A Nonlinear Bike Model for Purposes of Controller and Observer Design," *IFAC Proceedings Volumes (IFAC-PapersOnline)*, 2018.
- [2] N. H. Getz and J. E. Marsden, "Control for an autonomous bicycle," *International Conference on Robotics and Automation*, 1995.
- [3] A. Turnwald, M. Schäfer, and S. Liu, "Passivity-Based Trajectory Tracking Control for an Autonomous Bicycle," *IEEE, proceeding, IECON 2018*, 2018.
- [4] J. T. Betts, "Survey of Numerical Methods for Trajectory Optimization," *Journal of Guidance, Control, and Dynamics*, vol. 21, no. 2, pp. 193–207, 1998. [Online]. Available: <http://arc.aiaa.org/doi/10.2514/2.4231>

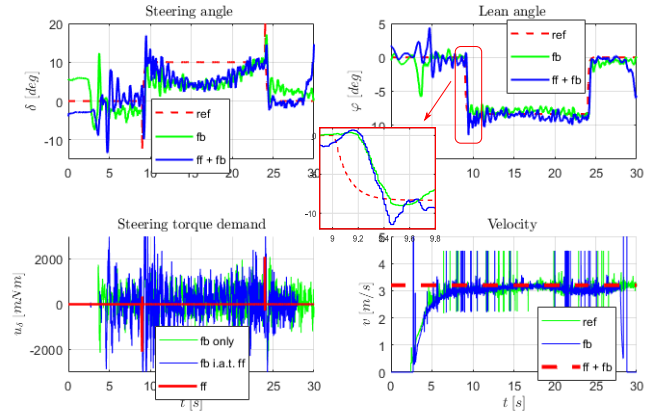


Fig. 7. Results for the trajectories created by the **naive** model

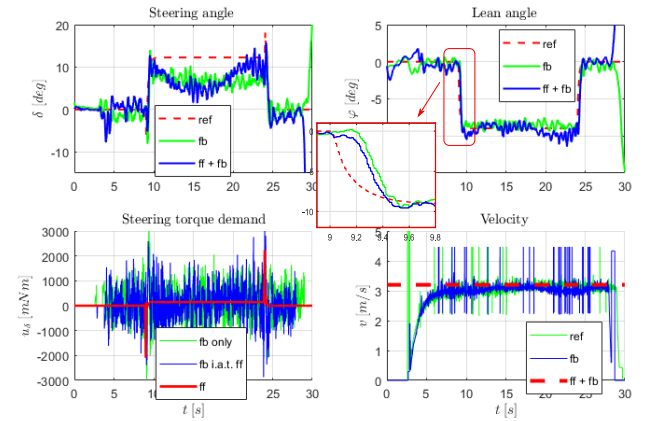


Fig. 8. Results for the trajectories created by the **trailed** model

- [5] M. P. Kelly, "An introduction to trajectory optimization: how to do your own direct collocation," *SIAM Review - Submitted*, 2016.
- [6] B. Paden, M. Cap, S. Z. Yong, D. Yershov, and E. Frazzoli, "A Survey of Motion Planning and Control Techniques for Self-Driving Urban Vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 1, no. 1, pp. 33–55, 2016.
- [7] J. P. Laumond, C. Morton, V. Pickert, and M. Armstrong, "Self-alignment torque as a source of energy recovery for hybrid electric trucks," *IEEE Transactions on Vehicular Technology*, vol. 63, no. 1, pp. 62–71, 2014.
- [8] T. Howard, M. Pivtoraiko, R. A. Knepper, and A. Kelly, "Model-predictive motion planning: Several key developments for autonomous mobile robots," *IEEE Robotics and Automation Magazine*, vol. 21, no. 1, pp. 64–73, 2014.
- [9] J. Yuan, H. Chen, F. Sun, and Y. Huang, "Trajectory planning and tracking control for autonomous bicycle robot," *Nonlinear Dynamics*, vol. 78, no. 1, pp. 421–431, 2014.
- [10] J. Hauser, A. Saccon, and R. Frezza, "Achievable motorcycle trajectories," *2004 43rd IEEE Conference on Decision and Control (CDC) (IEEE Cat. No.04CH37601)*, vol. 4, pp. 3944–3949 Vol.4, 2008.
- [11] A. Saccon, J. Hauser, and A. Beghi, "Trajectory exploration of a rigid motorcycle model," *IEEE Transactions on Control Systems Technology*, vol. 20, no. 2, pp. 424–437, 2012.
- [12] L. Keo, K. Yoshino, M. Kawaguchi, and M. Yamakita, "Experimental results for stabilizing of a bicycle with a flywheel balancer," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 6150–6155, 2011.
- [13] M. R. García, D. A. Mántaras, J. C. Álvarez, and D. B. F., "Stabilizing an Urban Semi-Autonomous Bicycle," *IEEE Access*, vol. 6, 2018.
- [14] P. Wang, J. Yi, T. Liu, and Y. Zhang, "Trajectory Tracking and Balance Control of an Autonomous Bikebot," *IEEE ICRA, Proceedings*, 2017.